

Chapter 6 | Exercise Answers

Basic SQL

Ramez Elmasri and Shamkant B. Navathe, Fundamentals of Database Systems, 7th edition.

Questions 6.1-6.16 follow the photographed chapter supplied for this course. Use the question numbers and printed page locators to read these answers beside your textbook.

These are original worked answers, not an official publisher solution manual. Open-ended designs state their assumptions. Full question text and photographed textbook figures are not reproduced.

Examples are identified as textbook data or original test data. SQL output tables are produced by executing the shown queries. SQLite-specific limitations are stated where they affect the result.

Publication of these answers does not expand the assigned sections or examination scope. Only taught and assigned material is assessed.

6.1. SQL tables and mathematical relations

Source: photographed textbook, printed p./pp. 233; explanations 220-222.

Relational term	SQL term	Important distinction
Relation	Table	A mathematical relation is a set; an SQL table may contain duplicate rows without suitable constraints.
Tuple	Row	A repeated output row need not represent the same base entity.
Attribute	Column	Column types and constraints describe allowed stored values.
Domain	Type / domain / CHECK	A base SQL type may be wider than the intended business domain.

Duplicates are allowed because eliminating them costs work and multiplicities can be meaningful, for example when two different employees have the same salary. Use DISTINCT when distinct projected values are intended; use keys when duplicate base entities must be prevented.

The question says Chapter 3, but the relational-model chapter in these photographed contents is Chapter 5. The answer follows the relational definitions, not that cross-reference.

6.2. SQL attribute types

Source: photographed textbook, printed p./pp. 233; explanations 212-214.

Family	Examples	Use
Exact numeric	INTEGER, SMALLINT, NUMERIC(p,s), DECIMAL(p,s)	Counts and fixed-scale amounts.
Approximate numeric	REAL, FLOAT, DOUBLE PRECISION	Approximate measurements.
Character	CHAR(n), VARCHAR(n)	Fixed-length or variable-length text.
Bit strings	BIT(n), BIT VARYING(n)	Sequences of bits.
Boolean	BOOLEAN	Logical values, with NULL where permitted.
Temporal	DATE, TIME, TIMESTAMP, INTERVAL	Dates, times, instants and durations.
Named domain / user-defined type	CREATE DOMAIN / CREATE TYPE	Reusable value definitions, where supported.

These are textbook SQL categories, not a claim that SQLite implements each declaration with standard semantics. SQLite uses storage classes and type affinity; dates below are ISO-format TEXT. DECIMAL declarations do not guarantee exact fixed-scale arithmetic in SQLite. Use integer cents for exact monetary calculations. CREATE DOMAIN is not supported.

6.3. Keys, foreign keys and referential actions

Source: photographed textbook, printed p./pp. 233; explanations 214-217.

```
CREATE TABLE PARENT(
  Id TEXT PRIMARY KEY NOT NULL);
CREATE TABLE CHILD(
  Id TEXT PRIMARY KEY NOT NULL,
  ParentId TEXT REFERENCES PARENT(Id)
  ON DELETE SET NULL ON UPDATE CASCADE);
```

Constraint / action	Effect
PRIMARY KEY + NOT NULL	Unique, present identifier; explicit NOT NULL also avoids SQLite's nullable non-integer primary-key exception.
FOREIGN KEY	Requires an existing referenced key unless the reference is allowed to be NULL.
RESTRICT / NO ACTION	Reject an operation that would leave a broken reference; timing can differ.
CASCADE	Propagate referenced-key updates, or delete dependent rows on parent deletion.
SET NULL	Replace the reference with NULL; it must permit NULL.
SET DEFAULT	Use the default; it must still satisfy the foreign key.

Enable PRAGMA foreign_keys = ON before starting a SQLite transaction. A default pointing to a deleted parent is not a valid repair.

6.4. Four clauses of a simple retrieval

Source: photographed textbook, printed p./pp. 233; explanations 217-229.

Clause	Contains	In the textbook template
SELECT	Columns, expressions or *; optionally DISTINCT.	Required
FROM	Source tables, joins and aliases.	Required
WHERE	Row conditions and join predicates.	Optional
ORDER BY	Sort expressions and ASC / DESC.	Optional

```
SELECT Fname,Lname  
FROM EMPLOYEE  
WHERE Dno=5  
ORDER BY Lname ASC;
```

This returns department 5 employees sorted by surname. Actual SQL dialects can also allow a constant query such as SELECT 1 without FROM; that exception does not change the textbook's table-query template.

6.5. Car-rental DDL

Source: photographed textbook, printed p./pp. 233; Exercise 5.14 p.203.

All seven relations are declared below. Quoted identifiers preserve the question's # character. Each REFERENCES clause is a foreign key. Key assumptions are one car per rental occurrence and one service per car, garage and date.

```
CREATE TABLE CUSTOMER(
  "Cust#" TEXT PRIMARY KEY NOT NULL, Cname TEXT, City TEXT);

CREATE TABLE CAR(
  "Car#" TEXT PRIMARY KEY NOT NULL, Year INTEGER, Model TEXT,
  Price NUMERIC CHECK(Price>=0), Depreciation NUMERIC,
  Last_service TEXT);

CREATE TABLE DRIVER(
  "Driver#" TEXT PRIMARY KEY NOT NULL, DName TEXT, DAddress TEXT);

CREATE TABLE GARAGE(
  "Garage#" TEXT PRIMARY KEY NOT NULL, GAddress TEXT, Owner_name TEXT);

CREATE TABLE RENTAL(
  "Rental#" TEXT PRIMARY KEY NOT NULL, Rdate TEXT,
  "Cust#" TEXT NOT NULL REFERENCES CUSTOMER("Cust#")
    ON DELETE RESTRICT ON UPDATE CASCADE,
  Time TEXT, Date TEXT, Hourly_rate NUMERIC CHECK(Hourly_rate>=0));

CREATE TABLE RENTAL_CAR(
  "Rental#" TEXT NOT NULL REFERENCES RENTAL("Rental#")
    ON DELETE RESTRICT ON UPDATE CASCADE,
  "Car#" TEXT NOT NULL REFERENCES CAR("Car#")
    ON DELETE RESTRICT ON UPDATE CASCADE,
  "Driver#" TEXT NOT NULL REFERENCES DRIVER("Driver#")
    ON DELETE RESTRICT ON UPDATE CASCADE,
  Start_time TEXT NOT NULL, End_time TEXT NOT NULL,
  Amount_received NUMERIC CHECK(Amount_received>=0),
  PRIMARY KEY("Rental#","Car#"), CHECK(End_time>Start_time));

CREATE TABLE SERVICING(
  "Garage#" TEXT NOT NULL REFERENCES GARAGE("Garage#")
    ON DELETE RESTRICT ON UPDATE CASCADE,
  "Car#" TEXT NOT NULL REFERENCES CAR("Car#")
    ON DELETE RESTRICT ON UPDATE CASCADE,
  Service_date TEXT NOT NULL,
  PRIMARY KEY("Garage#","Car#","Service_date));
```

These SQLite-tested statements use ISO date-time TEXT for Start_time and End_time so comparisons also work across midnight. Preserve historical rentals and services with ON DELETE RESTRICT; cascade identifier corrections. No-overlap scheduling and agreement between booked duration and charges need additional transaction-safe checks. The DDL does not claim to enforce those cross-row rules.

6.6. AIRLINE DDL

Source: photographed textbook, printed p./pp. 233; Figure 5.8 p.202.

The following nine tables retain the displayed composite keys. All twelve foreign keys are explicit. Dates and times use documented ISO-format TEXT in this SQLite implementation.

```
CREATE TABLE AIRPORT(
  Airport_code TEXT PRIMARY KEY NOT NULL, Name TEXT, City TEXT, State TEXT);

CREATE TABLE FLIGHT(
  Flight_number TEXT PRIMARY KEY NOT NULL, Airline TEXT, Weekdays TEXT);

CREATE TABLE AIRPLANE_TYPE(
  Airplane_type_name TEXT PRIMARY KEY NOT NULL,
  Max_seats INTEGER NOT NULL CHECK(Max_seats>0), Company TEXT);

CREATE TABLE AIRPLANE(
  Airplane_id TEXT PRIMARY KEY NOT NULL,
  Total_number_of_seats INTEGER NOT NULL CHECK(Total_number_of_seats>0),
  Airplane_type TEXT NOT NULL REFERENCES AIRPLANE_TYPE(Airplane_type_name)
  ON DELETE RESTRICT ON UPDATE CASCADE);

CREATE TABLE CAN_LAND(
  Airplane_type_name TEXT NOT NULL REFERENCES AIRPLANE_TYPE
  ON DELETE CASCADE ON UPDATE CASCADE,
  Airport_code TEXT NOT NULL REFERENCES AIRPORT
  ON DELETE CASCADE ON UPDATE CASCADE,
  PRIMARY KEY(Airplane_type_name,Airport_code));

CREATE TABLE FLIGHT_LEG(
  Flight_number TEXT NOT NULL REFERENCES FLIGHT
  ON DELETE RESTRICT ON UPDATE CASCADE,
  Leg_number INTEGER NOT NULL CHECK(Leg_number>0),
  Departure_airport_code TEXT NOT NULL REFERENCES AIRPORT
  ON DELETE RESTRICT ON UPDATE CASCADE,
  Scheduled_departure_time TEXT,
  Arrival_airport_code TEXT NOT NULL REFERENCES AIRPORT
  ON DELETE RESTRICT ON UPDATE CASCADE,
  Scheduled_arrival_time TEXT, PRIMARY KEY(Flight_number,Leg_number));

CREATE TABLE LEG_INSTANCE(
  Flight_number TEXT NOT NULL, Leg_number INTEGER NOT NULL,
  Date TEXT NOT NULL,
  Number_of_available_seats INTEGER NOT NULL
  CHECK(Number_of_available_seats>=0),
  Airplane_id TEXT NOT NULL REFERENCES AIRPLANE
  ON DELETE RESTRICT ON UPDATE CASCADE,
  Departure_airport_code TEXT NOT NULL REFERENCES AIRPORT
  ON DELETE RESTRICT ON UPDATE CASCADE,
  Departure_time TEXT, Arrival_airport_code TEXT NOT NULL REFERENCES AIRPORT
  ON DELETE RESTRICT ON UPDATE CASCADE,
  Arrival_time TEXT,
  PRIMARY KEY(Flight_number,Leg_number,Date),
  FOREIGN KEY(Flight_number,Leg_number) REFERENCES FLIGHT_LEG
  ON DELETE RESTRICT ON UPDATE CASCADE);

CREATE TABLE FARE(
  Flight_number TEXT NOT NULL REFERENCES FLIGHT
  ON DELETE RESTRICT ON UPDATE CASCADE,
  Fare_code TEXT NOT NULL, Amount NUMERIC CHECK(Amount>=0),
  Restrictions TEXT, PRIMARY KEY(Flight_number,Fare_code));
```

```
CREATE TABLE SEAT_RESERVATION(  
    Flight_number TEXT NOT NULL, Leg_number INTEGER NOT NULL,  
    Date TEXT NOT NULL, Seat_number TEXT NOT NULL,  
    Customer_name TEXT, Customer_phone TEXT,  
    PRIMARY KEY(Flight_number,Leg_number,Date,Seat_number),  
    FOREIGN KEY(Flight_number,Leg_number,Date) REFERENCES LEG_INSTANCE  
        ON DELETE RESTRICT ON UPDATE CASCADE);
```

Delete restrictions preserve flight, instance, fare and reservation records. CAN_LAND is a pure compatibility link and is removed when either parent is removed. Identifier corrections cascade.

Additional checks are needed for actual-plane capacity, type compatibility at the two actual airports, operating weekdays, time zones, seat labels and double-booked aircraft. These cross-table rules are not implemented by a simple CHECK. A flight leg's actual arrival may differ from its scheduled airport; requiring equality would be an extra policy.

6.7. Hotel queries

Source: photographed textbook, printed p./pp. 233.

Parts 1 and 2 are direct table selections. Part 3 is underspecified: a guest may live in New York or be booked into a New York hotel. Both interpretations are shown.

```
-- 1
SELECT * FROM Hotel ORDER BY hotelNo;
-- 2
SELECT * FROM Hotel WHERE hotelCity='New York' ORDER BY hotelNo;
-- 3: residence; assumes city text occurs in guestAddress
SELECT * FROM Guest WHERE guestAddress LIKE '%New York%'
ORDER BY lastName DESC,guestNo;

-- Alternative 3: has a booking at a New York hotel
SELECT G.* FROM Guest G
WHERE EXISTS (SELECT 1 FROM Booking B JOIN Hotel H
  ON H.hotelNo=B.hotelNo
  WHERE B.guestNo=G.guestNo AND H.hotelCity='New York')
ORDER BY G.lastName DESC,G.guestNo;
```

Original test data	Residence	Booking
Alex Young	New York	None
Blair Adams	Boston	H1, a New York hotel

The residence query returns Alex; the booking query returns Blair. A historical booking is not proof that the guest is in the city now. A current-stay query needs an explicit as-of date. Parsing a free-text address is only a simplifying assumption; a separate city field would be safer.

6.8. LIBRARY DDL

Source: photographed textbook, printed p./pp. 233-234; Figure 6.6 p.234.

Seven relations, with the keys underlined in the photographed figure. In particular, BOOK_LOANS has key (Book_id, Branch_id, Card_no); Date_out is not underlined.

```
CREATE TABLE PUBLISHER(
  Name TEXT PRIMARY KEY NOT NULL, Address TEXT, Phone TEXT);

CREATE TABLE BOOK(
  Book_id INTEGER PRIMARY KEY, Title TEXT NOT NULL,
  Publisher_name TEXT REFERENCES PUBLISHER(Name)
  ON DELETE SET NULL ON UPDATE CASCADE);

CREATE TABLE BOOK_AUTHORS(
  Book_id INTEGER NOT NULL REFERENCES BOOK
  ON DELETE CASCADE ON UPDATE CASCADE,
  Author_name TEXT NOT NULL, PRIMARY KEY(Book_id,Author_name));

CREATE TABLE LIBRARY_BRANCH(
  Branch_id INTEGER PRIMARY KEY, Branch_name TEXT, Address TEXT);

CREATE TABLE BORROWER(
  Card_no INTEGER PRIMARY KEY, Name TEXT, Address TEXT, Phone TEXT);

CREATE TABLE BOOK_COPIES(
  Book_id INTEGER NOT NULL REFERENCES BOOK
  ON DELETE RESTRICT ON UPDATE CASCADE,
  Branch_id INTEGER NOT NULL REFERENCES LIBRARY_BRANCH
  ON DELETE RESTRICT ON UPDATE CASCADE,
  No_of_copies INTEGER NOT NULL CHECK(No_of_copies>=0),
  PRIMARY KEY(Book_id,Branch_id));

CREATE TABLE BOOK_LOANS(
  Book_id INTEGER NOT NULL, Branch_id INTEGER NOT NULL,
  Card_no INTEGER NOT NULL REFERENCES BORROWER
  ON DELETE RESTRICT ON UPDATE CASCADE,
  Date_out TEXT NOT NULL, Due_date TEXT NOT NULL,
  PRIMARY KEY(Book_id,Branch_id,Card_no),
  FOREIGN KEY(Book_id,Branch_id) REFERENCES BOOK_COPIES
  ON DELETE RESTRICT ON UPDATE CASCADE,
  CHECK(Due_date>=Date_out));
```

The composite loan reference ensures the book is held by that same branch. Deleting a publisher sets its optional reference to NULL; deleting a book removes its author links, but is restricted while copies or loans require it. Loans and borrowers are preserved by delete restrictions.

The printed loan key cannot store two historical loans of the same book at the same branch by the same borrower. For a history system, add LoanId or Date_out to the key under an explicit revised requirement. Do not silently change the figure's key in an answer that claims to implement it.

6.9. How a DBMS enforces keys

Source: photographed textbook, printed p./pp. 233; explanations 214-217.

Operation	Necessary check
Insert or change a key	Reject NULL key components and search a unique index for duplicates.
Insert or change a foreign key	Find the matching parent key, unless the reference is allowed to be NULL.
Delete or change a referenced key	Find child references, then reject or apply the declared action.

Indexes make the usual lookup efficient. Child-key indexes help a parent update find referencing rows. Large cascades still require work proportional to the affected data.

The checks are straightforward conceptually but must be integrated with atomic transactions, concurrent writes, rollback and deferred constraints. An application's separate "check then insert" can race with another transaction and is not a substitute for database enforcement.

6.10. COMPANY queries with actual results

Source: photographed textbook, printed p./pp. 234; Figure 5.6 p.192.

Results use the photographed COMPANY state, not invented employee rows. Part (a) says salary greater than 3000, not 30000.

```
SELECT DISTINCT E.Fname,E.Lname
FROM EMPLOYEE E JOIN WORKS_ON W ON W.Essn=E.Ssn
JOIN PROJECT P ON P.Pnumber=W.Pno
WHERE E.Dno=5 AND E.Salary>3000 AND P.Pname='ProductZ'
ORDER BY E.Fname;
```

Observed result

Fname	Lname
Franklin	Wong
Ramesh	Narayan

```
SELECT Fname,Lname FROM EMPLOYEE
WHERE Address LIKE '%, Houston, TX' AND Super_ssn='333445555'
ORDER BY Fname;
```

Observed result

Fname	Lname
John	Smith
Joyce	English

For (b), Houston is read as the city in the displayed address and "under" as directly supervised by the specified Ssn. Ramesh lives in Humble and is excluded.

```
SELECT DISTINCT E.Fname,E.Lname
FROM EMPLOYEE E JOIN WORKS_ON W ON W.Essn=E.Ssn
JOIN PROJECT P ON P.Pnumber=W.Pno
WHERE P.Pname='Computerization' ORDER BY E.Fname;
```

Observed result

Fname	Lname
Ahmad	Jabbar
Alicia	Zelaya
Franklin	Wong

Part (c) spells the project Computerisation, but Figure 5.6 stores Computerization. The query uses the stored American spelling to answer the intended project question. A literal equality comparison with Computerisation would return no rows.

6.11. SQL for the eleven proposed updates

Source: photographed textbook, printed p./pp. 234; Exercise 5.11 pp.200-201.

Run each letter against a fresh Figure 5.6 database. Rejected statements are intentional constraint demonstrations, not code that should be made to pass by disabling foreign keys.

(a) Rejected under rejecting foreign-key actions. Missing supervisor 222244555; insert the real supervisor first or correct the reference.

```
INSERT INTO EMPLOYEE VALUES('Sophia','M','Wood','973442298',
    '1974-05-21','23 S Lamar Blvd, Austin, TX','F',62000,'222244555',5);
```

(b) Accepted. One project added. Key 4 is unused; department 4 exists.

```
INSERT INTO PROJECT VALUES('Sigma',4,'Austin',4);
```

(c) Accepted. One department added. The manager exists; manager membership in that same department is not a stated FK.

```
INSERT INTO DEPARTMENT VALUES
    ('Information Technology',2,'987987987','2007-10-01');
```

(d) Rejected under rejecting foreign-key actions. Both employee 777624972 and project 15 are missing.

```
INSERT INTO WORKS_ON VALUES('777624972',15,40.0);
```

(e) Accepted. One dependent added. Bdate is not part of the key and is nullable here.

```
INSERT INTO DEPENDENT VALUES('888665555','John','M',NULL,'Son');
```

(f) Accepted. One row removed: Abner.

```
DELETE FROM DEPENDENT WHERE Essn='987654321';
```

(g) Rejected under rejecting foreign-key actions. Referenced by four employees, three projects and three department locations.

```
DELETE FROM DEPARTMENT WHERE Dnumber=5;
```

(h) Accepted. Three assignments removed; the project itself remains.

```
DELETE FROM WORKS_ON WHERE Pno=30;
```

(i) Accepted. Three projects change. Department 1 exists.

```
UPDATE PROJECT SET Plocation='Houston',Dnum=1 WHERE Dnum=5;
```

(j) Accepted. One row matches; Borg already has no supervisor, so stored values do not change.

```
UPDATE EMPLOYEE SET Super_ssn=NULL WHERE Ssn='888665555';
```

(k) Rejected under rejecting foreign-key actions. Three WORKS_ON rows refer to 30. An approved ON UPDATE CASCADE would change their Pno to 40.

```
UPDATE PROJECT SET Pnumber=40 WHERE Pnumber=30;
```

For rejected statements, roll back before the next independent test. See Answer 5.11 for authorized repair alternatives and the Dname-length / NULL Hours source qualifications.

6.12. University retrievals

Source: photographed textbook, printed p./pp. 235; Figure 2.1 p.64.

Figure 2.1 supplies the Figure 1.2 schema. The displayed results below use original test rows, not the unavailable photographed Figure 1.2 state. The question asks for SQL, so no textbook result rows are assumed.

Original student	Class / Major	Sections / grades
Alex (17)	1 / MATH	101 A; 102 A
Blair (18)	2 / CS	101 B
Casey (19)	1 / MATH	101 NULL
Drew (20)	1 / MATH	No registrations
Emery (21)	1 / CS	101 A; 102 NULL

Section	Course / name / credits	Semester / year	Instructor
101	CS1 / Databases / 3	Fall / 2008	Anderson
102	MA1 / Algebra / 3	Fall / 2008	Hanif
103	CS1 / Databases / 3	Spring / 2009	Anderson

(a) Department comparison accepts either cs or CS in the supplied data.

```
SELECT Course_name FROM COURSE WHERE UPPER(Department)='CS';
```

Observed result

Course name
Databases

(b) Use the stored year representation consistently; this test fixture stores the four-digit year as TEXT.

```
SELECT C.Course_name,S.Instructor
FROM COURSE C JOIN SECTION S ON S.Course_number=C.Course_number
WHERE S.Semester='Fall' AND S.Year='2008'
ORDER BY C.Course_number;
```

Observed result

Course name	Instructor
Databases	Anderson
Algebra	Hanif

(c) Keep empty sections with LEFT JOIN. Count the non-NULL student key, not the outer-join placeholder.

```

SELECT S.Section_identifier,S.Course_number,S.Semester,S.Year,
       COUNT(G.Student_number) AS Students
FROM SECTION S LEFT JOIN GRADE_REPORT G
  ON G.Section_identifier=S.Section_identifier
WHERE S.Instructor='Anderson'
GROUP BY S.Section_identifier,S.Course_number,S.Semester,S.Year
ORDER BY S.Section_identifier;

```

Observed result

Section identifier	Course number	Semester	Year	Students
101	CS1	Fall	2008	4
103	CS1	Spring	2009	0

(d) Follow the explicitly printed Class = 1 despite the word "junior". Assume a completed course has a non-NULL grade.

```

SELECT T.Name,C.Course_name,C.Course_number,C.Credit_hours,
       S.Semester,S.Year,G.Grade
FROM STUDENT T JOIN GRADE_REPORT G
  ON G.Student_number=T.Student_number
JOIN SECTION S ON S.Section_identifier=G.Section_identifier
JOIN COURSE C ON C.Course_number=S.Course_number
WHERE T.Class=1 AND T.Major='MATH' AND G.Grade IS NOT NULL
ORDER BY T.Student_number,S.Section_identifier;

```

Observed result

Name	Course name	Course number	Credit hours	Semester	Year	Grade
Alex	Databases	CS1	3	Fall	2008	A
Alex	Algebra	MA1	3	Fall	2008	A

6.13. University insertions and update

Source: photographed textbook, printed p./pp. 235.

```
-- (a)
INSERT INTO COURSE(Course_name,Course_number,Credit_hours,Department)
VALUES('Financial Accounting','fac4390',5,'BUSINESS');
-- (b), after (a) so its course reference exists
INSERT INTO SECTION(Section_identifier,Course_number,Semester,Year,Instructor)
VALUES(145,'fac4390','Fall','17','Hanif');
-- (c)
INSERT INTO STUDENT(Name,Student_number,Class,Major)
VALUES('Robin',34,3,'BUSINESS');
-- (d)
UPDATE STUDENT SET Class=3
WHERE Student_number=17 AND Class=1;
```

Check on the original fixture	Observed result
Course fac4390	Inserted, 5 credits, BUSINESS
Section 145	Inserted and references fac4390
Student 34	Robin, class 3, BUSINESS
Student 17	Class changes from 1 to 3

Year is the literal 17 requested in part (b). A database standardized to four-digit years would need an explicit conversion rule. These statements assume the new keys are unused; a second execution correctly fails on duplicate keys. The Class=1 guard avoids changing an unexpected current state.

6.14. Insert one hotel and raise all room prices

Source: photographed textbook, printed p./pp. 235.

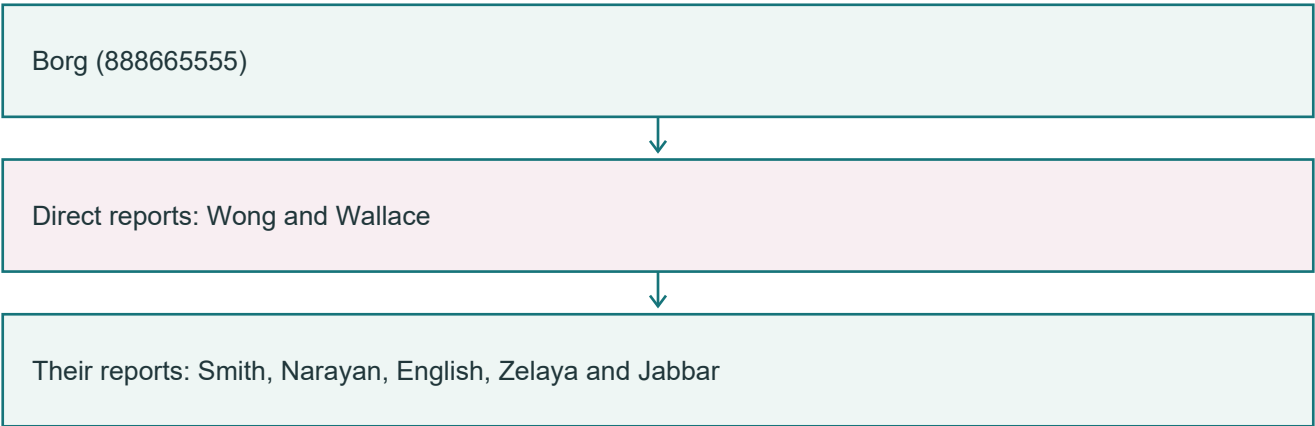
```
INSERT INTO Hotel(hotelNo,hotelName,hotelType,hotelAddress,  
                hotelCity,numRoom)  
VALUES('H1001','Hilton Hong Kong','Luxury','Beach Street',  
      'Hong Kong',200);  
UPDATE Room SET roomPrice=ROUND(roomPrice*1.15,2);
```

Original test room	Before	After
R1	100.00	115.00
R2	200.00	230.00
R3	80.00	92.00

There is no WHERE clause because the question says all rooms, not only rooms of H1001. The example assumes two decimal places. SQLite floating-point NUMERIC arithmetic is not an exact financial ledger; a production design should use an agreed exact monetary representation.

6.15. Deleting a supervisor with CASCADE

Source: photographed textbook, printed p./pp. 235; Figure 6.2 p.215.



(a) Considering only EMPSUPERFK, deleting Borg recursively reaches all eight employees, including Borg. ON DELETE CASCADE deletes referencing employee rows; it does not merely clear their Super_ssn.

```
DELETE FROM EMPLOYEE WHERE Lname='Borg';
```

Constraint environment	Verified outcome
Only the supervision foreign key	All 8 employee rows are removed.
Also retain Figure 6.2's department manager reference	Deletion fails: SET DEFAULT assigns manager 888665555, whose row is being removed. The original rows remain.

Other WORKS_ON and DEPENDENT references may also prevent the deletion unless their actions handle it. Thus "all employees are deleted" is the isolated cascade effect, not an unconditional successful outcome of the entire schema.

(b) SET NULL is usually more suitable for retaining employees when a supervisor leaves; it records that their supervisor is not currently assigned. RESTRICT can instead require reassignment first. CASCADE is appropriate only if the business rule truly requires deleting the subordinate entities. ON UPDATE CASCADE can still be useful for correcting an identifier.

6.16. A data copy is not a full backup

Source: photographed textbook, printed p./pp. 235.

```
CREATE TABLE EMPLOYEE_BACKUP AS
SELECT * FROM EMPLOYEE;
```

SQLite verification: eight rows are copied, and every copied value matches the original at creation time. Later changes to EMPLOYEE do not automatically change EMPLOYEE_BACKUP.

Copied by CREATE TABLE AS SELECT	Not automatically copied
Column names and query-result values	Primary/foreign keys, unique constraints, checks, indexes and triggers.

If the exercise requires the same constraints, create the backup table with explicit DDL first and then INSERT INTO EMPLOYEE_BACKUP SELECT A table in the same database file is not a disaster-recovery backup; loss of that file loses both tables.